CASE WESTERN RESERVE UNIVERISTY

Case Study #2 Report

Student names: *Augustus Kaufmann, Aaron Duda*

Course: *Machine Learning* – Professor: *Dr. Sanmukh Kuppannagari*
Due date: *May 6th, 2026*

Chosen Function and Varied Parameters

To select the best model suitable for determining the proper clustering of maritime vessel signals, we first considered using some of the algorithms in the scikit-learn module on clustering found at `https://scikit-learn.org/stable/modules/clustering.html`. We immediately eliminated heirarchical clustering methods since this is not what we are looking for. Additionally we do not use DBSCAN, HDBSCAN, OPTICS, BIRCH, or gaussian mixtures for a few reasons: (1) we want to cluster all samples and not label some as outliers, and (2) these algorithms provide additional capabilities which are not necessary for grouping maritime vessel signals. This left us with K-means, affinity propagation, and mean shift. For the sake of simplicity and the fact that the latter two are not scalable with respect to the number of samples, we decided to use the K-means algorithm.

To find the optimal number of samples, we set the following parameters, varied as specified.

1. `n_clusters` (Varied)

   This parameter represents the number of vessels in the data, and so this number along with the correct assignment of each sample to the correct vessel is the problem we are trying to solve. We do this by comparing the results of different values of this parameter and performing silhouette analysis to chose the optimal parameter.

2. `init`

   This parameter controls how we initialize the centroids. There is either a random approach, or an optimized greedy approach called `k-means++`. We set this parameter to `k-means++` since this is the default and performs faster than the traditional randomized approach.

3. `n_init`

   This parameter controls how many times to initialize and run the k-means algorithm, mitigating the risk of a bad initial assignment. we set this to 10 since we are optimizing for correct cluster assignment over speed of convergence.

4. `max_iter`

   This parameter controls how many iterations of k-means are allowed until the algorithm forcibly halts. We leave this at the default value 300.

5. `tol`

   This parameter is essentially the convergence threshold, and we leave this at the default value of 0.0001

6. `algorithm`

   This parameter determines which variant of k-means to run, either the classical formulation called `lloyd` or a version more efficient in some cases called `elkan`. We decided to go with the default `lloyd` algorithm since we are willing to sacrifice some speed for greater accuracy and simplicity.

Data Preprocessing

For data preprocessing, we follow a fairly simple procedure.

Data Imputation. First, we note that there was no data imputation required for the data set since all samples were intact from the start.

Training/Test Split. We split the data into training and test sets with a ratio of 20% test and 80% training. This test ratio is somewhat high since the amount of data points we have is small, so it may have an effect on the quality of our training since we are training on less data.

```
x_train, x_val, y_train, y_val = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
```

K-Fold Cross Validation. We then use k-fold cross validation with $k = 5$. This will split the data into 5 folds, and then when we train a model, we pick the configuration that minimizes the average error caused by the validation folds. Using k-fold improves our model's overall generalizability, because it lessens the effect of overfitting to the training samples or even overfitting to the validation samples if we had just set aside a static validation set.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=
   RANDOM_STATE)
```

Standardization. The final data preprocessing we do is standardizing the data for the Logistic and SVM models. This sets the model features on a comparable scale which helps with achieving accurate weights across all features. We do not standardize the data for the decision tree because feature scale does not matter when training the model. Decision trees do not learn weights, but instead the learn a tree based on the data features.

Hyperparameter Tuning

To tune our hyperparameters, we use randomized grid search over continuous distributions for each model's parameters. Instead of trying fixed combinations with grid

search, randomized search samples random combinations of hyperparameters which in theory should find the optimal solution faster. We first started out with grid search, but after switch to randomized grid search, we saw model improvements, such as decision tree accuracy jumping from about 80% to 89%. For continuous parameters such as C and γ, we sample from log-uniform distributions, and for the discrete parameters such as kernel type and tree criteria, we sample them uniformly from lists of options. Each candidate configuration is evaluated using 5-fold cross validation, and the configuration with the highest mean cross validation accuracy is selected as the best for that model. Thus, our method successfully finds the optimal hyperparameter combination.

```
# Example: SVM parameter distributions
{
    "classifier__kernel": ["rbf", "linear", "sigmoid"],
    "classifier__C": loguniform(0.1, 50),
    "classifier__gamma": ["scale", *loguniform(0.001, 1).rvs(10)],
}

search = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=parameter_grid,
    n_iter=50,
    scoring="accuracy",
    cv=cv,
    refit=True,
    random_state=RANDOM_STATE,
)
```

Best Model Configuration

For each model we compute the configuration which is the best as it relates to the mean cross validation accuracy and then compute a final accuracy using our hold-out test set. We chose the model which gave us the best final accuracy using the hold out test data. We decided to go with this method because the model which performs the bests on the hold-out test set has good generalizibilty since it had no way of knowing what the data was beforehand. The best model happend to be an SVM with the following configuration:

SVM with rbf kernel
Accuracy = 92.81%
C = 2.90
γ = 0.48
kernel = rbf

Observations. One important observation is that SVM performed best by a large margin. Its validation accuracy of 0.9281 was much higher than the Decision Tree validation accuracy of 0.8047 and the Logistic Regression validation accuracy of 0.7250.

Another observation is that the Decision Tree clearly outperformed Logistic Regression, but still did not come close to the performance of the SVM. This suggests that a simple linear model was too limited for this data, while a nonlinear model with a flexible decision boundary was more appropriate.